PNut-Term-TS

Downloader · Terminal · Debug Display

PNut-Term-TS User Guide

The Cross-Platform Downloader, Terminal, and Debug Display for the Propeller 2

November 2026

Version 1.0.1

Guide Organization

A practical guide to operating PNut-Term-TS

Getting Oriented & Using the GUI

- Where It Fits, and the Fork
- The Main Window
- Downloading & Running
- The Serial Terminal
- Debug Windows & Placement
- The Single-Step Debugger
- Menus, Settings, Devices
- Further Features

Headless & Reference

- Running Headless
- Ending a Run Cleanly
- The Log as Feedback Loop
- A Complete Automated Run
- Command-Line Reference
- Keyboard Shortcuts
- Troubleshooting
- Support & Resources



Contents

Copyright and License **6**

Part 1: Getting Oriented **7**

Chapter 1: Where PNut-Term-TS Fits **7**
The tool it belongs to 7

Chapter 2: Three Tools in One **9**
1. A downloader 9
2. A serial terminal — replacing Parallax Serial Terminal 9
3. A debug-window display — replacing PNut's, everywhere 10

Chapter 3: Two Ways to Run **11**
Headed — the interactive GUI 11
Headless — no windows, for automation 11

Chapter 4: Which Path Is Yours **12**

Part 2: Using the GUI **13**

Chapter 5: The Main Window **13**
The toolbar 14
The text-entry field 14
The terminal display 14
The status bar 14
The Debug Logger — a window onto the log, not the log itself 15

Chapter 6: Downloading and Running Your Program **17**
Two ways to load a program 17
RAM or flash 17
Resetting the P2 — and when it happens 17
Which PropPlug — you usually do not have to choose 18
Two baud rates, and why they are not the same number 18
The serial baud rate — you should not need to set it 19
The download baud rate — lower it when the link cannot keep up 21

Chapter 7: The Serial Terminal **23**
Reading output . . . 23
Typing back . . . 23
Look and feel . . . 23

Chapter 8: Debug Windows and Automatic Window Placement **24**
Windows open themselves . . . 24
Naming a display . . . 25
Automatic Window Placement . . . 25
Two more shared behaviors . . . 27

Chapter 9: The Single-Step Debugger **28**

Chapter 10: Menus, Settings, and Devices **29**
Menus differ by platform . . . 29
Settings and the hierarchy behind them . . . 29
PropPlug and device management . . . 31

Chapter 11: Further Features **33**
Recording and playback . . . 33
Performance monitoring . . . 33

Part 3: Headless and Automation **34**

Chapter 12: Running Headless **34**

Chapter 13: Ending a Run Cleanly **35**
Three ways to stop . . . 35
Exit codes — how a script reads the outcome . . . 35

Chapter 14: The Log Is Your Feedback Loop **37**
Program output stays clean . . . 37
What a log is named, and what it records . . . 37
The USB traffic log . . . 38

Chapter 15: A Complete Automated Run **39**
The agent in the loop . . . 39

Part 4: Reference **41**

Chapter 16: Command-Line Reference **41**

Chapter 17: Keyboard Shortcuts **43**

Chapter 18: Troubleshooting **44**

Chapter 19: Support and Resources **46**

Where the tools come from 46
Reporting a problem, or asking 46
Reading further 47

List of Figures

1.1 Where PNut-Term-TS sits in the Propeller 2 workflow. You work in the editor of your choice, and it drives both tools: `pnut-ts` builds the binary; PNut-Term-TS downloads it, starts it, and shows you what the chip sends back — as terminal text and as the debug windows your program draws to — while writing the same output to a log file. Only one line leads back to the editor, and it is the log: the windows are a place you watch, not a path your code travels. You close the loop yourself, by reading them and typing the next change. 8

2.1 The three tools PNut-Term-TS folds into one. 10

5.1 Startup opens two windows: the PNut-Term-TS main window (left) and the Debug Logger (right). 13

6.1 The two baud rates in User Settings — serial for the running program, download for the boot-loader exchange. 22

8.1 Automatic Window Placement: unpositioned windows fill a screen-sized grid center-out, top row first (numbers = open order; shown on the canonical 5-column × 3-row layout). The bottom row belongs to the main window and the debug logger — both are wider than one column, so they take the cells beside them too, leaving only the far-left cell for an eleventh window. 26

8.2 Several debug windows auto-placed on a real desktop: SCOPE, a TERM status read-out, and a Software-SPI LOGIC trace filling the top rows, with the main window and Debug Logger along the bottom in their reserved cells. 27

9.1 The single-step debugger window, hosted by PNut-Term-TS (shown here on macOS). 28

10.1 Preferences — the User Settings tab. 31

10.2 Preferences — the PropPlug Management tab. 32

15.1 Where PNut-Term-TS sits in the P2 agentic tool chain. With no windows to watch, the loop closes through the *log file* rather than through a direct line from the chip: everything the P2 sends comes back to PNut-Term-TS, which writes it to a log, and it is that log the agent reads. 40



Copyright and License

Copyright © 2026 Iron Sheep Productions, LLC.

PNut-Term-TS is © 2024–2026 Iron Sheep Productions, LLC and is licensed under the MIT License.

This user guide is licensed under the Creative Commons Attribution–ShareAlike 4.0 International License (CC BY-SA 4.0).

You are free to:

- **Share** — copy and redistribute the material in any medium or format
- **Adapt** — remix, transform, and build upon the material for any purpose, even commercially

Under the following terms:

- **Attribution** — You must give appropriate credit, provide a link to the license, and indicate if changes were made.
- **ShareAlike** — If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

To view the full license, visit: https://creativecommons.org/licenses/by-sa/4.0/

Trademarks

Parallax, Propeller, Spin, and the Parallax logo are trademarks of Parallax Inc. This license grants permissions under copyright only; it does not grant rights to use these trademarks, and adapted or redistributed copies must not imply endorsement by, or official status with, Iron Sheep Productions, LLC.

Part 1: Getting Oriented

Whoever you are and however you plan to use PNut-Term-TS, start here. This short part places the tool in the Propeller 2 workflow, shows you the three jobs it does, introduces the two ways you can run it, and then points you to the part of this guide written for *your* job. Everything after this part is split by how you work — so spend a few minutes here first, and you will land in the right place.

Chapter 1: Where PNut-Term-TS Fits

You have written a Propeller 2 program. You have compiled it. Now you want to put it on the chip and *see what it does* — the numbers it prints, the waveforms it draws, whether it behaves the way you expected. **PNut-Term-TS is the tool that closes that loop.** It downloads your compiled program to the P2, then receives and displays the `DEBUG()` output the program sends back over a USB serial connection.

The tool it belongs to

PNut-Term-TS does not work alone. It is the runtime end of a small, coherent set of tools for developing on the Propeller 2:

Tool	Its job
`pnut-ts`	The Spin2 / PASM2 compiler. Turns your source into a binary the P2 can run (and bakes in the debug settings).
`pnut-term-ts`	*This tool.* Downloads that binary to the P2 and shows you its `debug()` output.
Spin2 VS Code extension *(optional)*	Your editor, with Spin2 syntax and semantic highlighting. Search VS Code's Extensions view for `spin2` (Chapter 19).

Think of the first two as compile, then run-and-observe. `pnut-ts` produces the binary; **PNut-Term-TS is where you watch it come alive.**

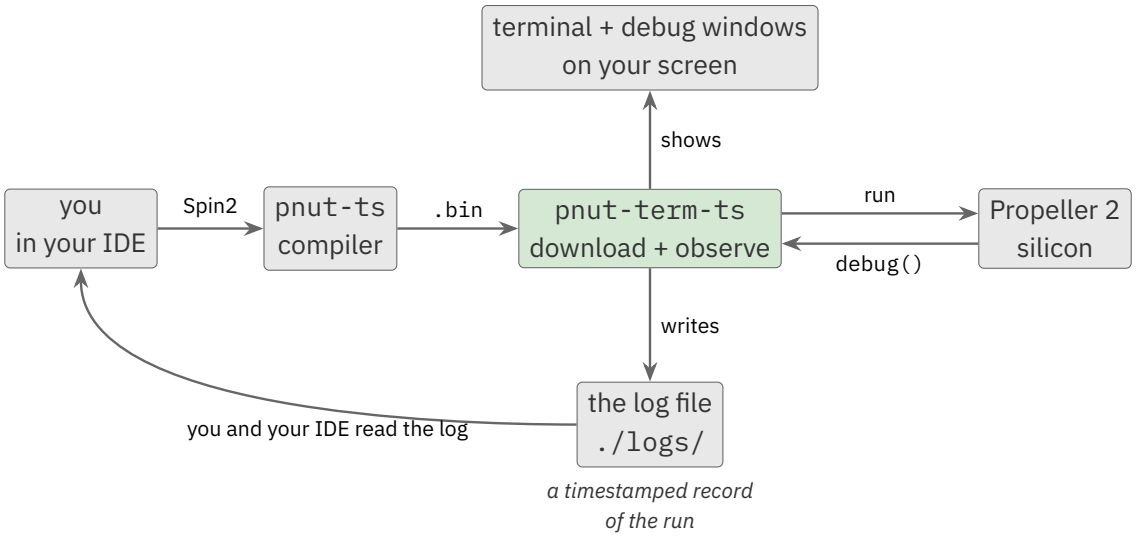


Figure 1.1. Where PNut-Term-TS sits in the Propeller 2 workflow. You work in the editor of your choice, and it drives both tools: `pnut-ts` builds the binary; PNut-Term-TS downloads it, starts it, and shows you what the chip sends back — as terminal text and as the debug windows your program draws to — while writing the same output to a log file. Only one line leads back to the editor, and it is the log: the windows are a place you watch, not a path your code travels. You close the loop yourself, by reading them and typing the next change.

Follow how that loop closes. **Everything the P2 sends comes back to PNut-Term-TS** — there is no second path off the chip. What arrives is shown to you as it happens, in the terminal and in the debug windows your program draws to, and the same output is written to a log file as it goes.

That log is not just a keepsake. It is the second way the run reaches you: you watch the windows while the program is live, then go back over the log to compare this run against the last one, or to find what scrolled past while you were looking elsewhere. The editor you work in can read it too — it is an ordinary text file. (Logs land next to the run, in `./logs/` relative to the folder you launched from, so the evidence stays beside the program that produced it. You can point them somewhere else if you would rather.)

Chapter 2: Three Tools in One

To understand PNut-Term-TS, start from what it replaces. It folds **three** jobs that used to need separate tools — or a specific operating system — into one program that runs the same way everywhere.

1. A downloader

It loads your compiled program onto the P2 and starts it running — either into **RAM** (fast, for the edit-run-edit loop of development) or into **flash** (so the program sticks and runs on power-up). It handles resetting the P2 into its loader for you.

It will download a binary from any compiler. Downloading is the P2's own boot protocol, so what built the image does not come into it. PNut, `pnut-ts`, FlexSpin and Spin Tools IDE all produce binaries this tool loads and starts, and so does anything else that can build for the P2 — if your toolchain produces a binary that downloads and runs, PNut-Term-TS can download and run it. You do not have to change compilers to use this tool.

What comes *back* follows a single rule, and it is a rule about the bytes on the wire rather than about your compiler:

What the P2 sends	Where it appears
debug output formed as *Parallax Spin2 Documentation v55* specifies	the debug log, and the debug windows it addresses
anything else written to the serial port	the terminal

Both halves matter. A program built **without** DEBUG is not a lesser case — every byte it writes still arrives and is shown in the terminal, exactly as any serial terminal would show it. Build **with** DEBUG and that output is recognised for what it is and routed onward, to the log and to the windows the commands name.

Which means the way for any compiler to be fully supported here is written down and public: emit what the Spin2 documentation specifies, and this tool will receive it. The one thing that is not carried on the wire is the serial baud rate — see Chapter 6, where compilers do differ.

2. A serial terminal — replacing Parallax Serial Terminal

Once your program is running, its `DEBUG()` text and any other serial output appear in a terminal window, and you can type back to the program. This is the job Parallax Serial Terminal did — now built in, and on every platform.

3. A debug-window display — replacing PNut's, everywhere

The P2's `DEBUG()` system can draw far more than text: oscilloscope traces, logic timing, plots, bitmaps, spectra, and an interactive single-step debugger. PNut can show these windows too — but only on Windows. **PNut-Term-TS renders the same windows on Windows, macOS, and Linux**, and it is where you *produce* the saved images and captures those windows can emit. That cross-platform reach is a large part of why the tool takes the form it does.

> **What the name tells you.** *PNut-Term-TS* reads as "PNut **Term**inal, written in **T**ype**S**cript." The *Terminal* is jobs 1 and 2; the *TypeScript* is why job 3 runs everywhere instead of on Windows alone.

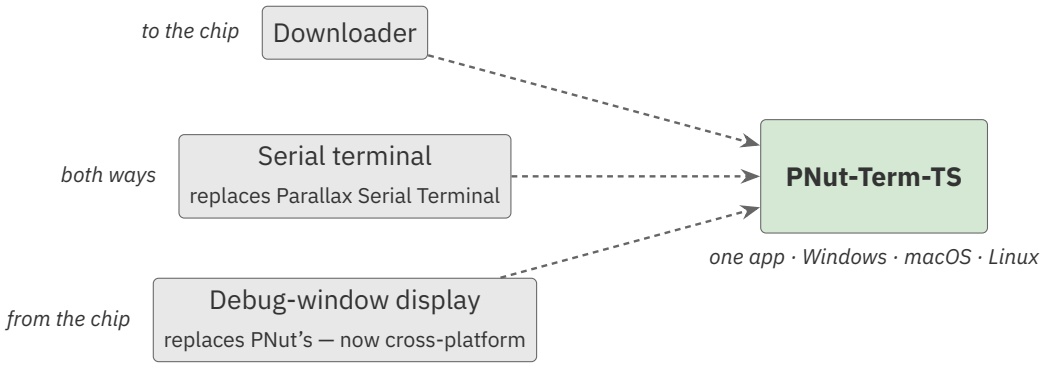


Figure 2.1. The three tools PNut-Term-TS folds into one.

The `DEBUG()` display windows themselves — what each one shows and how to author them from your Spin2 source — are the subject of the **P2 Debug Window Manual**, and the interactive single-step debugger has its own **P2 Single-Step Debugger Manual**. This guide is about the *tool that displays and produces them*; when you need the windows in depth, those manuals are where to go.

Chapter 3: Two Ways to Run

GUI and Headless

PNut-Term-TS runs in two fundamentally different ways, and the rest of this guide is organized around the difference. You choose between them by *how you launch the tool*.

Headed — the interactive GUI

Launch it normally and you get the full graphical application: a main window with a terminal, a toolbar to download and reset, and DEBUG windows that POP open on their own as your program draws to them. This is the mode you use **at your desk**, watching a P2 and reacting to what you see. You can start it on its own and load a program from the GUI, or name a program on the command line to download at once:

RUN THIS

```
pnut-term-ts                    # open the GUI, load a file from it
pnut-term-ts -r myprogram.bin   # open the GUI AND download this file
```

Headless — no windows, for automation

Launch it with `--headless` and there is **no graphical interface at all**. The tool downloads your program, captures everything the P2 sends to a timestamped log file, and exits on a signal you define. This is the mode built for **continuous integration pipelines, containers, and scripted test runs** on P2 hardware — anywhere a program, not a person, is watching.

RUN THIS

```
pnut-term-ts --headless -r TEST.bin --end-marker
```

With no windows, the log file is the only place the run can be seen: it is how whatever launched the tool finds out what the P2 did. The headless part of this guide covers it in depth.

Between these two poles are a few in-between modes — downloading from the command line but keeping the GUI, a headed “batch” run that exits when the program signals it is done, and an IDE-integration mode. They are covered where they belong, in the part for your workflow.

Whichever way you launch it, the tool’s help is built in: run `pnut-term-ts --help` at any time for the full list of options.

Chapter 4: Which Path Is Yours

The rest of this guide is written twice — once for each way of working — because the two jobs genuinely differ. Find yourself below and go there.

If you are...	...you want	Go to
At your desk, watching a P2 and reacting to what you see	The windows, the toolbar, recording a session, driving the single-step debugger by hand	**Part 2 — Using the GUI**
Automating P2 runs — CI, a container, or a script driving the hardware	Launching headless, ending a run cleanly, exit codes, and reading the log	**Part 3 — Headless and Automation**

You do not have to read the other path. What both paths share — the full command-line reference, keyboard shortcuts, the settings that shape either mode, and troubleshooting — lives in **Part 4 — Reference**, and both paths point you there when you need it.

If you are not sure yet, start with **Part 2**. The GUI shows you everything the tool can do in a form you can see, and much of what you learn there carries straight over to automating it later.

Part 2: Using the GUI

This part is for working **at your desk** — launching PNut-Term-TS as a graphical application, downloading a program, and watching it run in the terminal and the debug windows. If you are here to automate P2 runs instead, SKIP ahead to *Part 3 — Headless and Automation*; the two paths rejoin in *Part 4 — Reference*.

Chapter 5: The Main Window

When you launch PNut-Term-TS, **two windows open**: the **main window** and the **Debug Logger** (a window that shows you the run's log as it is written). These are the two windows the auto-layout keeps reserved spots for (Chapter 8); the other DEBUG windows — scopes, plots, and the rest — open on their own later, as your program draws to them.

This chapter covers both of them, starting with the main window: your home base for connecting, downloading, and reading text output. It has five areas, top to bottom — a **menu bar**, a **toolbar**, a **text-entry field**, the **terminal display**, and a **status bar**. The Debug Logger comes at the end of the chapter.

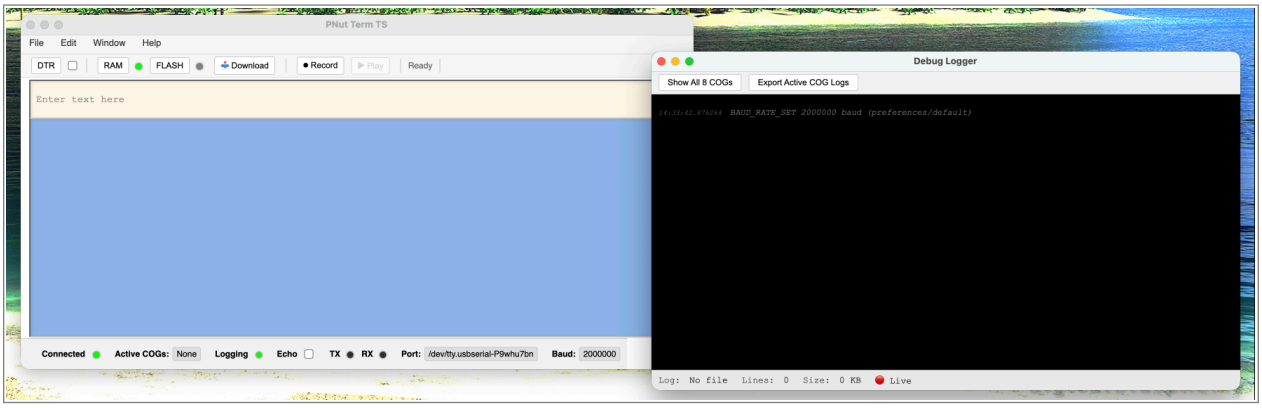


Figure 5.1. Startup opens two windows: the PNut-Term-TS main window (left) and the Debug Logger (right).

The toolbar

Left to right:

- **Reset control line** — a button labeled with the active reset line (**DTR** or **RTS**) and a checkbox. Click it to assert a reset on the P2. Which line a device uses is set per device in PropPlug Management (Chapter 10); this button just shows and fires it.
- **RAM** / **FLASH** — download the loaded binary into the P2's RAM or its flash and run it. The small indicator beside them is green for the active download target.
- **Download** (the tray icon) — pick a binary file to load.
- **Record** / **Play** — start or stop a recording, and play one back, with a status label beside them ("Ready", "Recording…"). Recording is a further feature covered briefly in Chapter 11.

The text-entry field

The field just below the toolbar sends a line of text to the running P2 when you press **Enter** — this is how you type back to a program that reads input. (During playback of a recording, a transport strip appears here instead — see Chapter 11.)

The terminal display

The large central area shows the text your program sends: its `DEBUG()` output and any other serial text. How it looks — terminal mode, color theme, font, and whether each line is tagged with the cog that produced it — is yours to set; Chapter 7 covers the terminal, and Chapter 10 the settings behind it.

The status bar

The status bar carries the at-a-glance state of the connection. On the **left**:

- **Connection indicator** — green when connected to a P2, amber when not.
- **Active cogs** — which cogs are currently driving debug output.
- **Logging indicator** — lit while the debug log is being written. (Recording is a separate thing, shown by the toolbar's status label.)

On the **right**:

- **Echo** — a checkbox that suppresses characters the P2 echoes back, so text you type is not shown twice.
- **TX / RX** — flash during serial transmit and receive.
- **Port** — the connected device's path.

- **Baud** — the active serial baud rate.

The active reset line (DTR or RTS) lives on the *toolbar* button, not here.

The Debug Logger — a window onto the log, not the log itself

The other window that opened at startup is the **Debug Logger**, and it is worth separating two things that share its name:

- **the log file** — the durable record on disk, written to the log directory;
- **the Debug Logger window** — a view of that file, live, as it fills.

The file is the record. The window is how you watch it.

Closing the window does not stop logging

Close the Debug Logger and you close a window. **The log keeps recording.** The file stays open, every line still reaches it, and it notes when the window was closed and when it came back — so a gap in your attention is never a gap in the record.

- **Window → Show Log** reopens it. It attaches to the **same** file rather than starting a new one, and repaints the recent history so you are not staring at a blank pane.
- **Window → Hide Log** closes it. The menu entry names whichever action applies.

The log is ended by the *run*, not by the window. Three things end one: a P2 reset and the start of a download each rotate to a fresh file, and quitting closes it.

A **cog** window works the same way. Closing one stops that window; that cog's log keeps recording.

Reading it

- The view **follows live data**. Scroll up to pause it and read; the view stays where you put it. Click **↓ Follow Live Data** to catch up again.
- Bytes you sent are marked `[TX]`, with control characters spelled out as `<cr>` and `<lf>`.
- A P2 reset writes a session marker, clears the view, and rotates the file.
- System messages, errors, warnings, binary and hex fallbacks, and debugger output are colour-coded.
- **Show All 8 Cogs** opens the full set of cog windows; **Export Active Cog Logs** writes the current cog logs out to files.
- The status bar shows the log's filename, its line count, its size, and whether the view is live or paused.

How much history the window keeps is set by **History Lines** in Preferences (default 1000). That is the window's memory, not the file's — the file keeps everything either way.

When output arrives faster than the window can draw

Under a heavy stream the window stops trying to draw every line, skips ahead to stay current, and tells you so:

```
... 450 line(s) NOT shown — display fell behind;
the log file has every line ...
```

This is not data loss. Only the drawing was reduced, and only while it was behind; the marker itself is never written to the file. At ordinary rates you will not see it.

It also points at the cheapest fix if the application feels sluggish under a flood: the Debug Logger is the most expensive window to draw, so **Hide Log** buys back the most, and it costs you nothing in the record.

Chapter 6: Downloading and Running Your Program

The core loop at your desk is: hand PNut-Term-TS a compiled program, tell it where to put it on the P2, and watch it run.

Two ways to load a program

There are two ways to hand PNut-Term-TS your compiled `.bin`, and you can mix them freely:

- **Interactively (standalone).** Launch PNut-Term-TS on its own, with no file — it opens ready and waiting. Click **Download** on the toolbar to choose a binary, then send it to the P2 with the **RAM** or **FLASH** button. Open the app once and load, and re-load, programs from the GUI as you work.
- **From the command line.** Name the binary when you launch, and PNut-Term-TS downloads and runs it straight away, keeping the GUI open:

RUN THIS

```
pnut-term-ts -r myprogram.bin      # download to RAM AND run
pnut-term-ts -f myprogram.bin      # download to FLASH AND run
```

RAM or flash

Either way you load it, you choose *where* the program goes on the P2:

- **RAM** (`-r`, or the toolbar **RAM** button) — the program is loaded and run immediately, but does not survive a power cycle. This is the fast path for the edit-compile-run loop of development.
- **FLASH** (`-f`, or the toolbar **FLASH** button) — the program is written to the P2's flash, so it runs on its own every time the board powers up.

You use one or the other — `-r` and `-f` cannot be combined. PNut-Term-TS resets the P2 into its loader at the right moment, downloads the program, and begins capturing debug output.

Resetting the P2 — and when it happens

To start your program cleanly, the P2 has to be reset. Whether that happens the moment PNut-Term-TS connects is governed by one preference, **Reset P2 on App Startup** (Chapter 10), and it reflects two different ways of working:

- **On** (the default) — reset on connect. Use this during development, so every run starts from a known state.
- **Off** — do not reset; attach to a program that is *already running*. Use this to monitor a board that is already doing its job.

The reset itself is asserted over one of two control lines — **DTR** or **RTS**. Parallax PropPlugs and most FTDI adapters use **DTR**; some clones need **RTS**. The line is remembered per device (Chapter 10), and you can override it for one session with `--rts`.

Which PropPlug — you usually do not have to choose

With a single PropPlug connected there is nothing to decide. PNut-Term-TS finds the one device and uses it, with no flag from you — and that is how most runs go.

It only becomes a question when **more than one** USB serial device is present: two boards on the bench, or yesterday's plug still in the hub. Then you say which one you mean with `-p`, giving either the device path or its serial number — a case-insensitive partial match is enough, so the first few characters usually do.

To find those serial numbers, ask:

RUN THIS

```
pnut-term-ts -n                          # list connected devices
pnut-term-ts -p P9cektn7 -r myprogram.bin  # then name the one you want
```

`-n` (`--dvcnodes`) lists the USB serial devices PNut-Term-TS can see and exits without touching the P2, so it is safe to run at any time. It is also the first thing to try when a board is not detected at all. Add `-m` if you want every FTDI device listed rather than PropPlugs alone.

If you reach for the same plug every day, you need not pass `-p` at all: set a **Default PropPlug** in **User Settings** for all your projects, or in **Project Settings** for one. Chapter 10 gives the full order these resolve in, and is also where you give a plug a friendly name — so that a serial number becomes something you can recognise.

Two baud rates, and why they are not the same number

A run has two phases — **downloading** your program, then **watching it run** — and they do not work the same way. PNut-Term-TS keeps a separate rate for each, because the two phases put the burden on opposite ends of the wire.

Downloading: the P2 adapts to you. The P2's boot ROM listens with *auto-baud* detection — it measures the timing of the first character you send and matches it. You never have to discover what the chip wants, because the chip discovers what you are using. PNut-Term-TS downloads at

2,000,000 bits per second, the fastest rate that detection reaches, and most people never think about it again.

Running: you adapt to the P2. Once your program is running there is no auto-baud. The program transmits at whatever rate its own code chose, and PNut-Term-TS has to be listening at that same rate or the text is unreadable.

That asymmetry is the whole reason for two settings — and for one of them being almost invisible.

Two rates, not three. The serial rate carries your program's `DEBUG()` output *and* anything you type at the terminal, because those are not two connections: they are one serial link, in two directions. There is no separate "terminal rate" to find and no third number to keep in step. Downloading is the one job with a rate of its own, and it has one because it is the one job where the P2 is listening rather than talking.

The serial baud rate — you should not need to set it

When you download a binary that PNut-Term-TS recognises, **it reads the rate out of the image itself** and listens at exactly that rate. PNut and `pnut-ts` write the value in — including when your source sets its own rate:

```
CON  DEBUG_BAUD = 921600   ' picked up automatically
```

If your source says nothing, the P2 toolchain defaults to **2,000,000** bits per second, and so does PNut-Term-TS. Either way, downloading a binary sets the rate for you, with no flag from you.

This is the one place in the download path where toolchains differ, so it is worth being plain about it. **PNut-Term-TS auto-detects a PNut or `pnut-ts` image** and takes the serial rate from it, which is why downloading one settles the question with no flag from you.

A binary it does not recognise as PNut or `pnut-ts` downloads and runs exactly the same way — that half does not depend on your compiler at all. What it does not do is hand over a rate. So whatever that program was built with, **you tell the tool the rate**: with `-b` for the session, or with the **Serial Baud Rate** preference if it is a board you come back to. This is as true of a program built *without* DEBUG as one built with it — plain serial text still has to be decoded at the rate it was sent, and nothing in an unrecognised image says what that rate is.

So `-b` (`--baud`) is the answer in three situations:

- **Attaching to a P2 that is already running** — no download, so there is no image to read a rate out of.
- **A debug build from any other toolchain** — the download works; only the rate has to come from you.

- **A program that just writes to the serial port** — no DEBUG involved at all, and the rate is whatever its own code set.

  The flag used to be spelled `--debugbaud`. That spelling is **deprecated** but still accepted, so existing scripts keep running. The old name was misleading: this rate carries *everything* the P2 sends back — `DEBUG()` output and plain serial text alike — so it is now `--baud`.

Either way you have two ways to say it: `-b` for one session, or the **Serial Baud Rate** preference if you would rather set it once and forget it.

That preference has a *scope*, which is what keeps it off your command line for good. Set it in **User Settings** and it applies to every project you open. Set it in **Project Settings** and it overrides that default for the current project only — so a board that needs an unusual rate can carry it without following you into your other work. Chapter 10 has the full order in which these resolve.

If you pass `-b` and it disagrees with the binary you are downloading, PNut-Term-TS warns you — the P2 will transmit at its own compiled rate regardless, and the mismatch would make the output unreadable:

```
WARNING: --baud 115200 disagrees with this binary's
compiled DEBUG_BAUD (2000000). The P2 will transmit at
2000000 — expect unreadable output. Drop --baud to use
the binary's rate.
```

`DEBUG_BAUD` there is the field's name inside the image, not a second setting. When text comes out garbled, the first thing to try is *dropping* `-b`.

The accepted range is 300 to 20,000,000, and its two ends mean quite different things — neither of them a statement about how fast your link will actually go.

The floor is framing, not speed. PNut-Term-TS configures 8N1 exclusively (Chapter 7), and 300 baud is the lowest rate it can reach at all: every slower historic rate needs framing this tool does not produce — the 110-baud Teletype wanted two stop bits for its carriage, and Baudot below that is a five-bit code entirely.

The ceiling is a **corruption guard**, not a capability claim. When PNut-Term-TS takes a rate out of a binary, that value has to be sanity-checked — a damaged image can carry anything at all in that field, and past 20,000,000 a number has stopped being a baud rate and started being a symptom. It is not a promise that your machine can drive 20 Mbaud.

What *can* be said about speed is narrower, and more useful: **2,000,000 is the highest rate this app has been verified to carry**, measured on hardware without loss. Ask for more and PNut-Term-TS accepts it, tries it, and warns you — because above that line nobody has run the experiment. It marks the edge of the evidence, not the edge of the capability.

So the honest position is the one the warning states: **unverified above 2 Mbaud.** It may carry your stream perfectly. It may drop data. Nobody knows yet — and the warning asks you to report what you observe, which is how the real ceiling for each platform will eventually get established.

If it *does* go wrong, know what to look for, because it will not announce itself. A host that cannot keep up does not garble the text — that is what a *mismatched* rate does. It loses pieces of it: the stream still reads as ordinary, well-formed output, with lines simply absent. That is a failure that looks like a bug in your program long before it looks like a baud rate.

The download baud rate — lower it when the link cannot keep up

The download rate governs **the boot-loader exchange and nothing else** — the conversation that gets your program onto the chip, over before your program starts. It is the rate you are least likely to touch, and the one worth knowing about when a download misbehaves.

PNut-Term-TS downloads at **2,000,000** bits per second by default. That is not an arbitrary choice: it is the top of the range the P2's boot ROM can lock onto, so it is both the fastest download available and one you never have to configure. The chip meets you wherever you are.

What the P2 can detect and what your *cable* can carry are two different questions. A long lead, a marginal USB-serial adapter, or a clone that will not clock 2 Mbaud can leave the download unable to complete — and because the chip simply never locks on, there is nothing to read on screen. That is when you lower it:

RUN THIS

```
pnut-term-ts --downloadbaud 921600 -r myprogram.bin
```

As with the serial rate, you can make it stick instead: **Download Baud Rate** in User Settings for every project, or in Project Settings for one — useful when a particular bench setup always needs a slower download.

The accepted range is 9600 to 2,000,000, and PNut-Term-TS refuses anything outside it rather than trying. The bounds are the P2's, not ours: they are the window its auto-baud detection can lock onto. Outside that window the chip never responds and the download simply never finishes, with nothing to tell you why — so refusing the number is kinder than accepting it and hanging.

Lowering the download rate does nothing to your program's output. The two rates are independent: one governs getting the program *onto* the chip, the other governs reading what it says once it is running. A slow download still yields a full-speed conversation.

Both rates sit together in Preferences, each with the tool's own one-line summary of what it governs — the same division of labour this chapter has been describing, in the place you will actually set it (Chapter 10 covers the dialog itself).

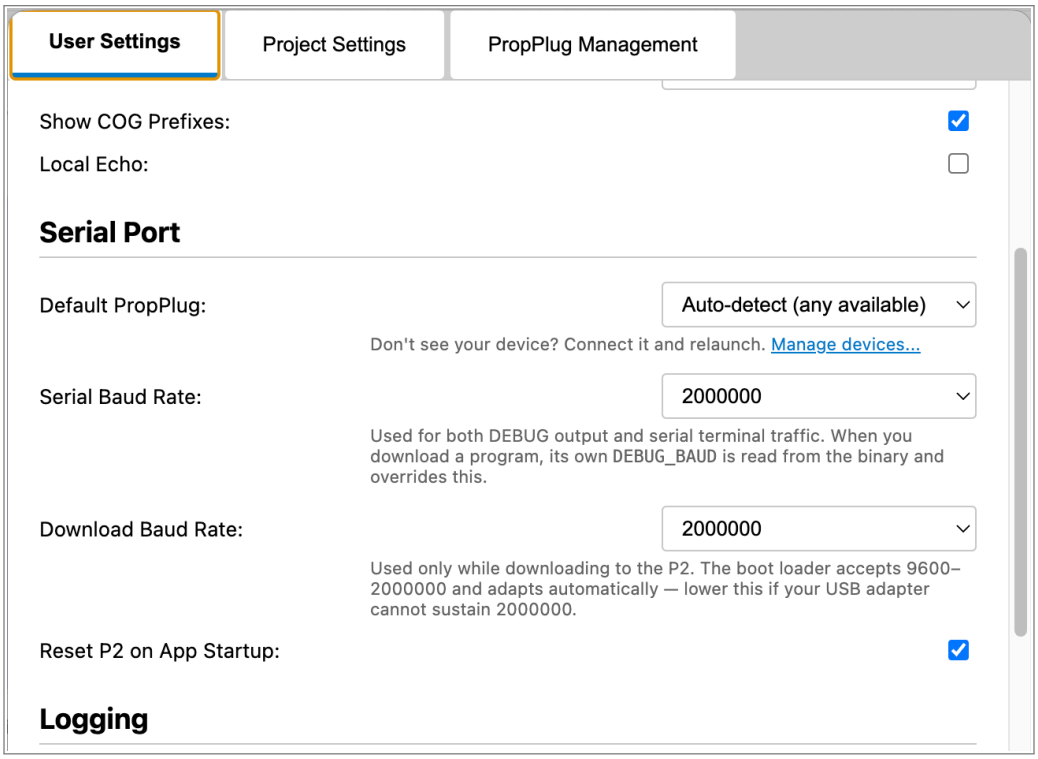


Figure 6.1. The two baud rates in User Settings — serial for the running program, download for the boot-loader exchange.

Chapter 7: The Serial Terminal

Once your program runs, its text appears in the terminal, and you can type back. This is the job Parallax Serial Terminal used to do — now built in, and the same on every platform.

The line is 8N1 — eight data bits, no parity, one stop bit. That is the framing the P2's own serial code uses and the only framing PNut-Term-TS speaks; there is no setting for it. It is worth knowing because it is the one mismatch a baud rate cannot rescue: if you point the tool at a device that talks 7E1, or at something old enough to want two stop bits, no rate will make the text readable. Everything in the P2 world is 8N1, so this matters only when you attach PNut-Term-TS to something that is not a P2.

Reading output

Each line the P2 sends is shown as it arrives. If **Show Cog Prefixes** is on (the default), each line is tagged with the cog that produced it, so you can tell overlapping output apart. You can search what has scrolled past with **Find** (`Ctrl+F`), and clear the display with **Clear Terminal**.

Typing back

Type into the text-entry field and press **Enter** to send a line to the running program. If the P2 echoes your characters back, tick the **Echo** box in the status bar so they are not shown twice.

Look and feel

The terminal's appearance is set in Preferences (Chapter 10); the choices that shape day-to-day reading are:

Setting	Choices	Default
Terminal Mode	PST, ANSI	PST
Color Theme	Green / White / Amber on Black	Amber on Black
Font Size	10–24	14
Font Family	Default, Parallax, IBM 3270 (+ Green / Amber)	Default
Show Cog Prefixes	on / off	on
Local Echo	on / off	off

Chapter 8: Debug Windows and Automatic Window Placement

The P2's `DEBUG()` system draws far more than text. Ask for a scope trace, a logic-timing view, a plot, a bitmap, and PNut-Term-TS opens a separate window for each — an oscilloscope, a logic analyzer, and so on, live on Windows, macOS, and Linux alike.

Windows open themselves

There is **no menu command to open a debug window.** A window appears the moment your program sends a `DEBUG()` display directive for it, and it is the directive that decides what the window shows. The window types are:

Window	Shows
TERM	A text terminal
BITMAP	A pixel / image display
PLOT	General X/Y plotting, shapes, sprites
SCOPE	Oscilloscope-style waveforms
SCOPE_XY	X-versus-Y (Lissajous / vector) display
LOGIC	Logic-analyzer timing
FFT	Frequency spectrum
SPECTRO	Spectrogram / waterfall
MIDI	MIDI keyboard / messages
Cog logger	Per-cog debug message log
Debugger	The interactive single-step debugger (Chapter 9)

> **Where to learn the windows themselves.** *What* each window displays and *how you author it* from your Spin2 source — the `DEBUG()` display directives and their parameters — are the subject of the **P2 Debug Window Manual** and the Parallax P2 DEBUG documentation. This guide covers the windows only as the tool *presents* them; when you want a window in depth, that manual is where to go.

Naming a display

You name a display in the `DEBUG()` directive that creates it — `` `PLOT MyPlot ... `` — and every later update addresses that window **by name**. The name is the display's only address on the wire, so it cannot be a word the display language is already using for something else.

Rule	Detail
Character set	Start with a letter or _, then letters, digits and _
Not a reserved word	No display type (`PLOT`, `TERM`, `SCOPE`, ...) and no directive keyword (`TRACE`, `SET`, `LINE`, `TITLE`, `WINDOW`, `CLOSE`, `RED`, `GRAY`, ...)
Case-insensitive	`trace`, `Trace` and `TRACE` are one name — the window still shows your spelling
Unique among open displays	Closing a display frees its name for reuse
30 characters	Longer names are shortened to 30, so two names that differ only after the 30th character become the same name

The reserved words are the **display language's own**, not Spin2's. `spin2` is a perfectly legal display name. So are `traces`, `my_trace` and `plotter`, which merely contain a reserved word without being one.

> **A bad name stops the run.** PNut-Term-TS reports the problem and exits with **code 4** (Chapter 13) rather than leaving you with a window that never appears. This is a deliberate difference from PNut, which discards the display statement **silently** — same program, no window, no message, and nothing to search for. Renaming the display in your P2 program and re-running is the whole fix.

Automatic Window Placement

A P2 program can name a screen position for each window with a `POS` directive, but it does not have to. **A window with no `POS` is placed for you automatically** — PNut-Term-TS lays the whole set out as a tidy *dashboard* instead of stacking every window on the same spot. (Windows that *do* carry a `POS` are put exactly where they ask.)

The dashboard is a grid **sized to your display**: the height sets how many rows, the width how many columns. A typical 1920×1080 screen gives a **3×3** grid; a 2560×1440 or a 4K display gives more cells (5×3 or 5×4); an ultra-wide gives more columns still, up to seven. There is always an odd number of columns, so a true center column exists to build around.

Windows fill the grid in a fixed **center-out** order: the first window takes the top-center cell, the next two flank it, and the arrangement widens outward and works its way down, row by row, stay-

ing balanced left-to-right as it goes. If you open more windows than the grid holds, the extras cascade from the top-left.

The bottom row is not really yours. The **main window** and the **debug logger** live there, and both are wider than a single column — so each of them also claims the cell beside it, which is what a window too wide for its cell always does. Between them they take the rest of the bottom row; on a typical screen they even overlap each other, as you can see in Figure 5.1. In practice that leaves the far-left cell as the only bottom slot an auto-placed window can land in.

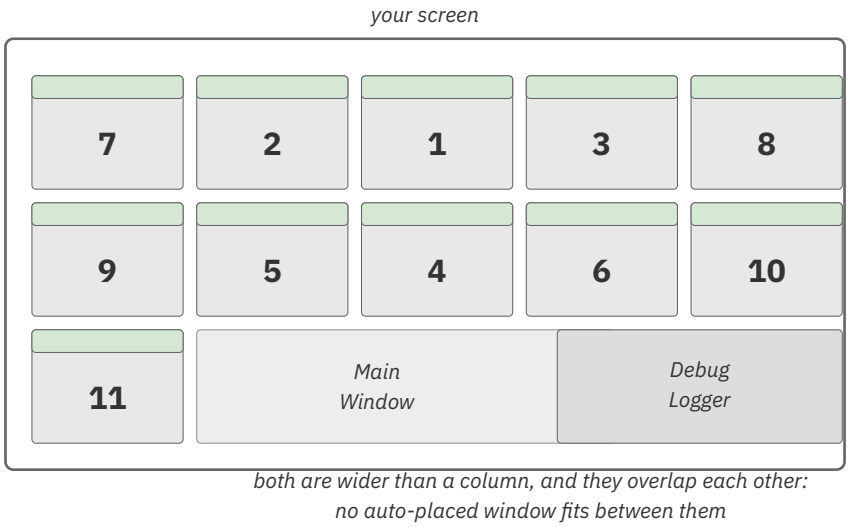


Figure 8.1. Automatic Window Placement: unpositioned windows fill a screen-sized grid center-out, top row first (numbers = open order; shown on the canonical 5-column × 3-row layout). The bottom row belongs to the main window and the debug logger — both are wider than one column, so they take the cells beside them too, leaving only the far-left cell for an eleventh window.

> **One thing to expect on a 1920-wide screen.** The fill *order* is defined against the five-column layout above, but the *pixel* positions are computed from the columns your display actually gets — and 1920 wide gives you three. The two outermost positions then fall off the right-hand edge of the work area and get clamped back to it, so those later windows stack near the right edge rather than tiling cleanly. It is a known constraint of the current placer, not something you have done wrong. Displays 2000 px and wider get five columns and lay out exactly as drawn. If you want a specific arrangement on a 1920-wide screen, that is a good moment to reach for `POS`.

The result is that you can throw a handful of `DEBUG()` displays on screen and get a readable dashboard with no `POS` directives at all. And when you *do* want to pin a window to an exact spot, you do not have to guess coordinates: **drag a display window and its title bar shows its live `x, y` position** as it moves — read the numbers off and bake them into a `POS` directive in your source. (The cog logger, the debugger, and the message-log windows do not show this readout; they are placed by their own rules.)

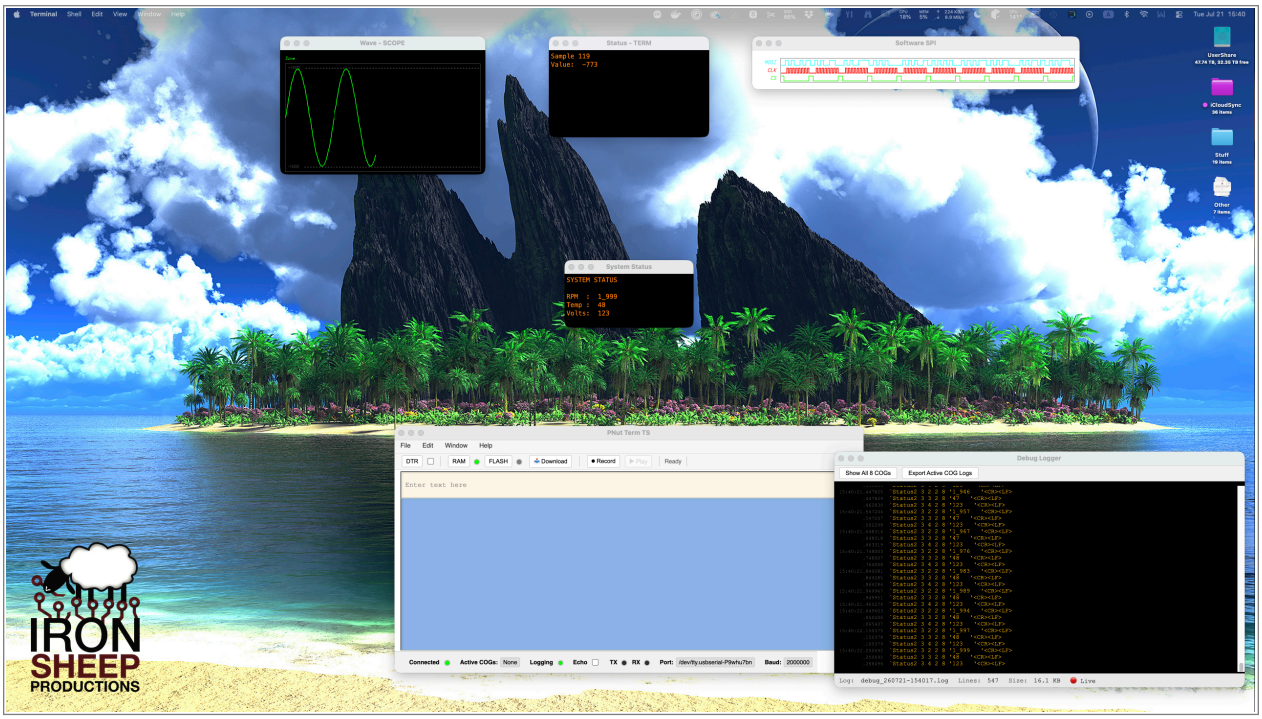


Figure 8.2. Several debug windows auto-placed on a real desktop: SCOPE, a TERM status readout, and a Software-SPI LOGIC trace filling the top rows, with the main window and Debug Logger along the bottom in their reserved cells.

Two more shared behaviors

- **SAVE** — a `DEBUG() SAVE` (or `SAVE WINDOW`) directive writes a window's current image out to a bitmap file. This is how you *produce* the captures the debug windows can emit.
- **Input back to the P2** — a window can forward mouse and key input to the running program, when the program asked for it with the `PC_MOUSE` / `PC_KEY` mechanisms.

Chapter 9: The Single-Step Debugger

One of the windows PNut-Term-TS can display is special: the **interactive single-step debugger**. When your program is compiled with debugging enabled and reaches a `DEBUG` statement, execution can pause and this window appears — showing registers, memory, flags, and the program counter, and letting you step, run, set breakpoints, and watch state as your PASM2 code executes.

PNut-Term-TS is what **renders that debugger window and carries your interaction with it** — the keystrokes and clicks that step and break — to the P2 and back. Bringing this interface to macOS and Linux, not just Windows, is a large part of why the tool exists.

> **Using the debugger is its own subject.** How to turn on debugging, read each region of the debugger window, set breakpoints, and drive a debugging session is covered fully in the **P2 Single-Step Debugger Manual**. This guide's part is only that the window is *here*, cross-platform, and interactive.

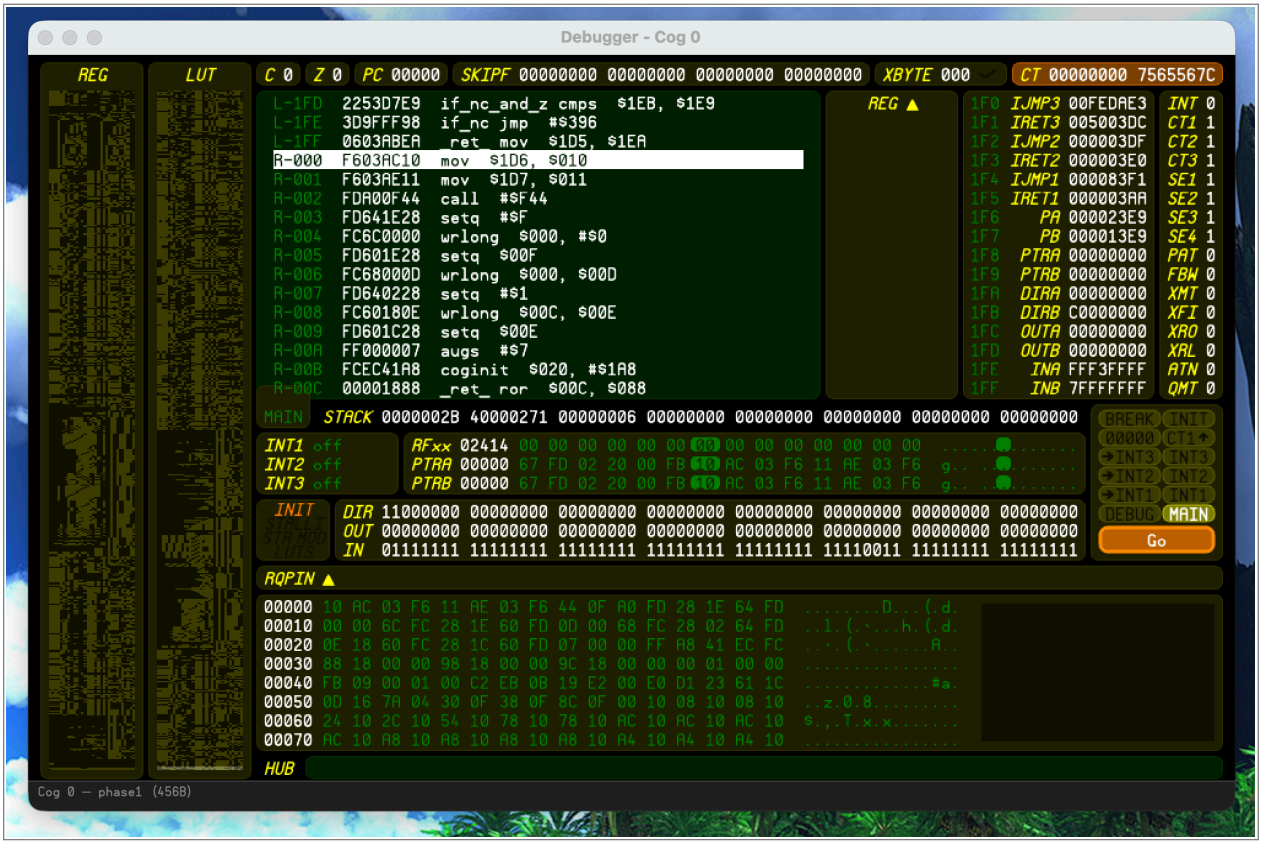


Figure 9.1. The single-step debugger window, hosted by PNut-Term-TS (shown here on macOS).

Chapter 10: Menus, Settings, and Devices

This chapter collects the GUI's configuration: the menus, the settings that shape every run, and the management of the USB devices you connect through.

Menus differ by platform

The menu bar inside the window is the application's menu, on every platform — File, Edit, Window and Help, the same four on Windows, Linux and macOS alike. What differs between platforms is the accelerator key: `Cmd` on macOS, `Ctrl` on Windows and Linux.

Menu	Items
File	New / Open / Save Recording; Select PropPlug; Start Recording (`Ctrl`+R); Stop Recording; Playback Recording (`Ctrl`+P); Exit (`Ctrl`+Q)
Edit	Cut / Copy / Paste; Find… (`Ctrl`+F); Clear Terminal; Preferences… (`Ctrl`+`,`)
Window	Show Log / Hide Log; Performance Monitor; Show All Windows; Hide All Windows
Help	Documentation (F1); About PNut-Term-TS

macOS shows one thing extra: the **native application menu** in the system menu bar at the top of the screen, where a Mac user expects to find it. It carries the standard macOS items only — About, Preferences… (`Cmd+,`), Hide and Show All, Quit, the clipboard commands, and Minimize / Close / Zoom / Bring All to Front. It is not a second copy of the application's menus, and nothing is missing from the Mac because of it: anything not in that short list is in the in-window menu bar, the same as everywhere else.

The **Window** menu's first entry names whichever action is available — **Show Log** when the Debug Logger window is closed, **Hide Log** when it is open. Closing that window does not stop logging; see Chapter 5.

Settings and the hierarchy behind them

Open settings with **Edit → Preferences…** (`Ctrl+,` or `Cmd+,` on macOS). Settings resolve in priority order, most specific first:

1. **Project settings** — overrides scoped to the current project directory.
2. **User settings** — your per-machine defaults.
3. **Application defaults** — the built-in baseline.

A value named on the **command line** beats all three, for that run only.

The two baud rates each resolve down that same ladder, independently — changing one never moves the other. The serial rate has one extra rung, because a recognised binary can *state* the rate its output will use:

	Serial baud	**Download baud**
Carries	`debug()` output and terminal traffic	the boot-loader exchange only
Command line	`-b`, `--baud`	`--downloadbaud`
Preference	Serial Baud Rate	Download Baud Rate
Resolves	command line → the binary → project → user → default	command line → project → user → default
Default	2,000,000, or the binary's `DEBUG_BAUD` when you download	2,000,000
Limits	300 – 20,000,000	9600 – 2,000,000
Warns	above 2,000,000	never

Nothing inside a binary can say what rate it should have been *downloaded* at — you would have to be reading it already to find out — which is why the download rate has no such rung. The preference dropdowns offer the common rates; the command line accepts any value in range.

The dialog has three tabs. The **User Settings** tab holds your machine-wide defaults:

Group	**Setting**	**Default**
Terminal	Mode / Theme / Font size / Font family / Cog prefixes / Local echo	PST · Amber on Black · 14 · Default · on · off
Serial Port	Default PropPlug / Serial Baud Rate / Download Baud Rate / Reset P2 on App Startup	Auto-detect · 2000000 · 2000000 · on
Logging	Log Directory / Auto-Save Debug Output / Enable USB Traffic Logging	`./logs/` · on · off
Recordings	Recordings Directory	`./recordings/`
Debug Logger	History Lines (100–10000)	1000

A P2 reset always starts a **new** log file — that boundary is what makes a log readable as a single run. Logs are not size-capped.

The **Project Settings** tab carries the same controls, each with an **override** checkbox: only the boxes you tick override your user defaults for the current project, and the rest show and inherit your global values.

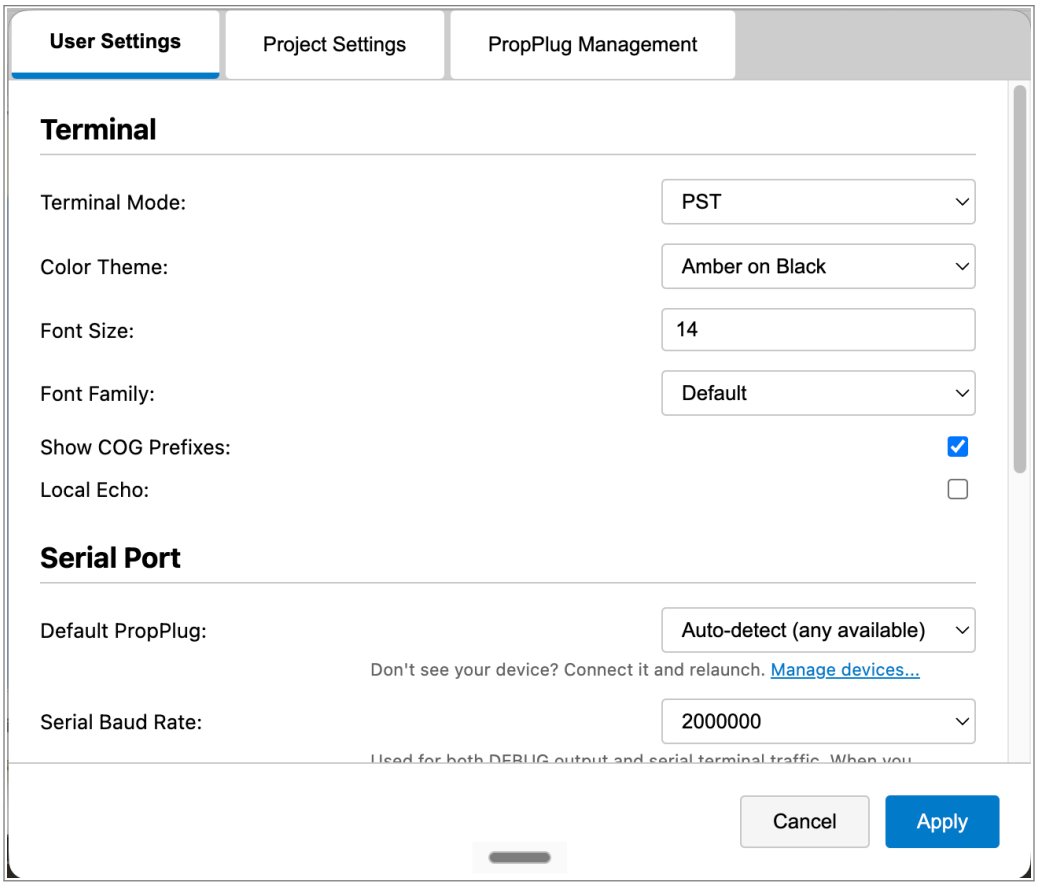


Figure 10.1. Preferences — the User Settings tab.

PropPlug and device management

PNut-Term-TS remembers every USB serial device it has seen and lets you name it and set its reset line. New devices are added automatically when they enumerate, with the Parallax-standard **DTR** control line.

Open the **PropPlug Management** tab to manage them. The known-devices table shows each device's serial number, friendly name, control line, and last-used time. Select a row to give it a **Friendly Name** (e.g. "Workbench Plug"), choose its **Control Line** (DTR or RTS), view its VID/PID, and **Save Changes** or **Delete Device**.

With exactly one device connected there is nothing to resolve — PNut-Term-TS uses it, and most runs never get further than that (Chapter 6). The order below is what settles it when **several** are connected, most specific first:

1. A command-line `-p <device>` — an exact match, or a case-insensitive partial match on the path or serial number. Run `pnut-term-ts -n` to list the connected devices and their serial numbers.
2. A project device override, if one is set.

3. Your user default.
4. Auto-detect — used when exactly one device is connected. This is the ordinary case, reached whenever you have said nothing more specific.
5. Otherwise, you are asked to choose (or the run errors if nothing matches).

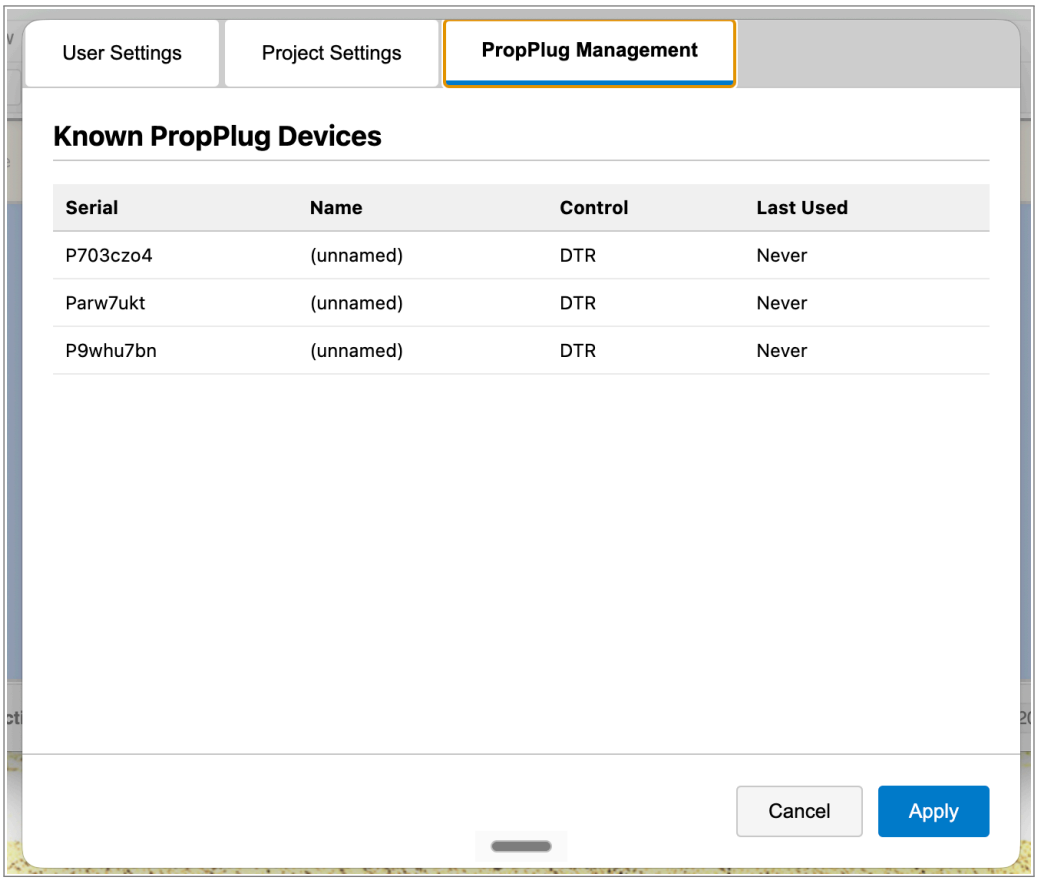


Figure 10.2. Preferences — the PropPlug Management tab.

Chapter 11: Further Features

Two capabilities sit outside the everyday download-and-watch loop. They are documented here — briefly, and deliberately out of the main flow — because one is present but not yet exhaustively exercised, and the other is aimed at development and diagnostics rather than routine use. Reach for them if they fit your need, but do not build a workflow on them yet.

Recording and playback

PNut-Term-TS can capture a whole DEBUG session to a **`.p2rec`** file and replay it later, timing and all — useful for regression testing, for sharing a reproduction, or for studying a run offline. Start and stop a recording from **File → Start Recording** (`Ctrl+R`) and **File → Stop Recording**; replay one with **File → Open Recording...** (`Ctrl+P`), which reproduces the captured stream — including its timing — driving the debug windows exactly as the live run did. Recordings land in your Recordings Directory (default `./recordings/`).

> This feature has had only limited testing so far. Treat it as experimental: it is here if it helps you, but it is not yet a foundation to build on.

Performance monitoring

Window → Performance Monitor opens a view of the serial-to-window data path — throughput, buffer usage, queue depth, and message counts — so you can see whether the tool is keeping up at high baud rates. It is primarily a **developer and diagnostic** aid: reach for it if you suspect the pipeline is falling behind (buffer usage climbing toward full), and otherwise you can leave it closed.

Part 3: Headless and Automation

This part is for running PNut-Term-TS where **no person is watching** — a continuous-integration pipeline, a container, or an AI coding assistant running programs on P2 silicon and reading back the results. There is no graphical interface here; the **log file is how the run is seen**. If you are working at your desk, *Part 2 — Using the GUI* is your part; both rejoin in *Part 4 — Reference*.

Chapter 12: Running Headless

Add `--headless` and PNut-Term-TS runs with **no windows at all**. It downloads your program, captures everything the P2 sends to a timestamped log file, and runs until you tell it to stop.

RUN THIS

```
pnut-term-ts --headless -r TEST.bin -p P9cektn7
```

Everything you know about downloading from Part 2 still applies — `-r` versus `-f`, the reset control line, both baud rates — because the download path is the same. What changes is that there are no DEBUG windows and no terminal: the program's output goes to the log instead of the screen.

Between the full GUI and fully headless sit two in-between modes you may reach for:

- **Headed batch** — a normal GUI run that *exits on its own* when the program signals it is done, draining any pending window SAVEs and logs first. Use it for scripted capture that still needs on-screen rendering:

  RUN THIS

  ```
  pnut-term-ts -r gen.bin --exit-on-end-session -p P9cektn7
  ```

- **IDE integration** — a minimal UI meant to be driven by an editor such as the Spin2 VS Code extension:

  RUN THIS

  ```
  pnut-term-ts --ide -p P9cektn7          # DTR reset
  pnut-term-ts --ide --rts -p P9cektn7    # RTS reset
  ```

Chapter 13: Ending a Run Cleanly

A person closes the GUI when they are done. An automated run needs a defined way to stop — and a way to tell success from failure without looking. PNut-Term-TS gives you both.

Three ways to stop

A headless run ends on any of:

- a **signal** — `Ctrl+C` or `SIGTERM`;
- a **timeout** — `--timeout <seconds>`, which stops the run after a fixed time;
- an **end marker** — `--end-marker`, which stops the run when a phrase appears in the output.

RUN THIS

```
pnut-term-ts --headless -r TEST.bin --end-marker    # stop on the marker
pnut-term-ts --headless -r TEST.bin --timeout 60    # stop after 60s
```

By default the end marker matches either `END_SESSION` or `DEBUG_END_SESSION` (a case-sensitive match anywhere in the output), so a program that prints one of those when it finishes will end the run by itself. Give `--end-marker "PHRASE"` to match your own phrase instead. The same marker mechanism drives the headed-batch `--exit-on-end-session` mode from Chapter 12.

Exit codes — how a script reads the outcome

When PNut-Term-TS exits it returns a code, and **the codes are the same whether you ran with the GUI or headless**, so a launching script can branch on `$?` identically either way:

Code	Meaning
0	Clean exit — all SAVEs and logs flushed, and the captured log is complete
1	Port / device error — the command was valid; the hardware was not there, or it stopped responding mid-run
2	Bad command line — nothing ran
3	Download failed
4	Fatal DEBUG display error in your P2 program — a display name the tool cannot use, or the same name twice (Chapter 8)
124	Headless `--timeout` expired
125	The log may be incomplete — the shutdown drain ran long, or output was lost while writing

Code 0 is a promise about the log, not about the shutdown. It says the capture you are about to read is complete: every line the P2 sent reached the file. That is the thing a script actually

needs to know before it parses the log, which is why the code asserts it. Code 125 is its opposite number — the run may have finished perfectly well and the *record* still be short.

One thing worth knowing about code 2: the command line is checked **before anything runs**. If an option is wrong, PNut-Term-TS reports *every* problem with it at once and exits — no device is touched, no download attempted, no window opened.

Code 4 comes from your P2 program rather than from the command line, so it can only appear once a program is running. It means a `DEBUG()` directive named a display something the tool cannot address — see *Naming a display* in Chapter 8.

Chapter 14: The Log Is Your Feedback Loop

In headless mode the log file is not a side effect — **it is the whole point.** It is how an automated caller, or an assistant reading back the behavior of code it just wrote, sees what the P2 did. So it is worth understanding what the log holds and what it deliberately keeps out.

Program output stays clean

The log's first job is to carry your program's output — its `DEBUG()` text — **complete and unpolluted**, because that text is what the automated reader is there to see. PNut-Term-TS keeps that stream clean and adds only a thin layer of run narrative alongside it: window-placement notices, download start / success / failure, the baud and reset lines, and any directive errors or warnings (the ones that help you fix a bad `DEBUG()` directive).

That narrative records **events, not the machinery that carries them out**. One reset of the P2 is one line; the several control-line transitions that perform it are not, and neither are port handles opening and closing or a channel being reopened. Those are transport diagnostics — real, and available when you want them under `--diag-serial` (below), but kept out of an ordinary run so they cannot crowd out the output you came for. Errors are the exception and always appear: a port that will not open is something you need to see whether or not you asked for detail.

What a log is named, and what it records

Each run writes a timestamped file to the log directory (by default `./logs/`, next to where you launched the run, so logs land beside the program that produced them). The names are:

Log	File name
Debug log (headed runs)	`debug_YYMMDD-HHMMSS.log`
Headless log	`headless_YYMMDD-HHMMSS.log`
USB traffic log	`usb-traffic_YYMMDD-HHMMSS.log`

Every log begins with a banner that records **which build produced it** — `PNut-Term-TS: vX.Y.Z` — because a captured log is often kept as regression evidence, and evidence has to say what version it came from.

A log's life is the run's life. Two things start a fresh file — **a P2 reset** and **the start of a download** — and quitting closes the one in hand, so each file reads as a single clean run. Closing a window is not one of them; the Debug Logger is a view of the file, not the file itself (Chapter 5).

The USB traffic log

For the times you need to see the raw bytes on the wire, turn on the **USB traffic log** with `-u` (`--log-usb-trfc`). It captures the runtime byte conversation — everything exchanged once the P2 is running:

- the `DEBUG()` stream coming *from* the P2, and
- everything the host sends *back* — typed terminal input, `PC_KEY` / `PC_MOUSE` forwarding, and single-step-debugger responses.

Direction coverage depends on the mode, and this is by design: a **headed** run captures both directions, while a **headless** run is **receive-only** — nothing in the headless path sends bytes to the P2 after the download, so there is no transmit side to record.

Two things the USB log deliberately does **not** contain: the reset and handshake that *get* the program running, and the downloaded binary image itself — logging the image would bury the interesting bytes under hundreds of kilobytes of hex. One consequence is worth remembering: **an empty USB traffic log is meaningful.** It means the P2 never produced any runtime traffic — most often because the download failed and the program never started.

That case is exactly what `--diag-serial` is for. It logs the serial-channel detail the USB log leaves out — the download handshake steps among them — which is where the answer lives when the program never started and the runtime log has nothing to show you.

When what you want is throughput rather than content, add `--usb-counts-only` alongside `-u`. It records receive timestamps and byte counts and skips the hex dump, which keeps a long capture readable and small.

Chapter 15: A Complete Automated Run

Putting Part 3 together, here is the shape of a hardware-in-the-loop run an assistant or a CI job can drive end to end: **download, run, stop on a marker, then read the log.**

Have your program print a known phrase when it finishes — say a line containing `DEBUG_END_SESSION` — so the run ends by itself. Then:

RUN THIS

```
pnut-term-ts --headless -r build/TEST.bin -p P9cektn7 --end-marker
```

When the marker appears, PNut-Term-TS flushes and exits. The caller then does two things:

1. **Checks the exit code.** `0` means a clean finish; a non-zero code (Chapter 13) says what went wrong — `3` for a failed download, `124` for a timeout, and so on — without any need to read the log.
2. **Reads the log.** The freshest `headless_*.log` in `./logs/` holds the program's `DEBUG()` output — the actual behavior of the code under test, ready to be parsed, compared against an expected result, or fed back to the assistant that wrote the program.

That download → run → marker → read cycle is the loop that lets a program, rather than a person, develop and verify P2 code on silicon.

The agent in the loop

That cycle is what makes this tool the runtime end of an *agentic* P2 workflow — an assistant that writes P2 code, compiles it, runs it on silicon, and reads the log back to decide what to do next. Seen that way, the tool chain of Chapter 1 gains one more member, because an assistant needs a source for what it is writing about:

Tool	Its job
P2KB MCP	Serves the P2 knowledge base — the instruction set, the language, the silicon — to an assistant that is writing P2 code.
`pnut-ts`	The Spin2 / PASM2 compiler. Turns that source into a binary the P2 can run.
`pnut-term-ts`	*This tool.* Downloads the binary, runs it, and writes back what the P2 said.

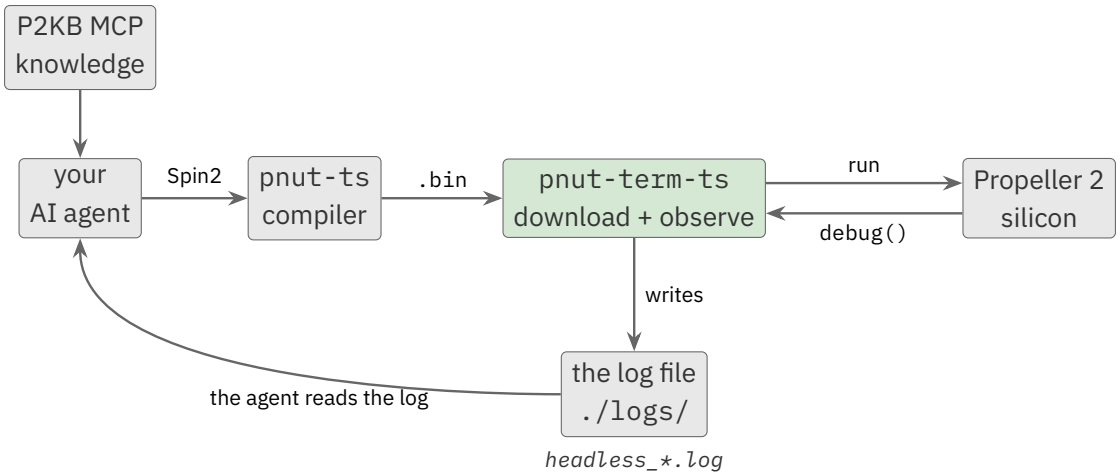


Figure 15.1. Where PNut-Term-TS sits in the P2 agentic tool chain. With no windows to watch, the loop closes through the *log file* rather than through a direct line from the chip: everything the P2 sends comes back to PNut-Term-TS, which writes it to a log, and it is that log the agent reads.

Follow how that loop closes, because the shape of it is the whole point. The agent does not read the P2 directly. **Everything the P2 sends comes back to PNut-Term-TS, which writes it to a log file in the `logs` folder — and it is that file the agent reads.** At your desk that log is one of two ways back — the one you turn to after the run, alongside what you were watching live on screen. Here there is no screen, so the log is not one path among several: it *is* the return path, and it is the reason a program can take the place of the person who used to watch.

That agent-in-the-loop way of working is the subject of **The P2 Architect's Guide, Part 3**, which names this very tool chain — `pnut-ts`, `pnut-term-ts`, and the Knowledge Base — as what lets a hosted agent close the write-compile-run-read loop on its own. This guide is the operating manual for the tool that makes it possible.

Part 4: Reference

The material both paths point into: the full command-line reference, the keyboard shortcuts, troubleshooting, and where to go for more.

Chapter 16: Command-Line Reference

This same list is built into the tool — run `pnut-term-ts --help` for it at any time.

RUN THIS

```
pnut-term-ts [options]
```

Table 16.1.

Option	Long form	Argument	Description
`-r`	`--ram`	file	Download the file to **RAM** and run
`-f`	`--flash`	file	Download the file to **FLASH** and run
`-p`	`--plug`	device	Use the device at `<device>` (path or serial; partial match OK). Auto-detects if exactly one is present
`-b`	`--baud`	rate	The **serial** baud rate — `debug()` output and terminal traffic. 300–20000000; warns above 2000000, the highest verified (Chapter 6). `--debugbaud` still accepted, deprecated
	`--downloadbaud`	rate	The **download** baud rate — the boot-loader exchange only. 9600–2000000; lower it when the link cannot hold 2 Mbaud (Chapter 6)
`-n`	`--dvcnodes`		List detected USB serial devices and exit
`-m`	`--match-vendor-only`		With `-n`, list any FTDI device, not just PropPlugs
`-d`	`--debug`		Emit detailed diagnostic messages
`-v`	`--verbose`		Emit verbose messages
`-q`	`--quiet`		Suppress the banner and non-error text

Continued on next page

Table 16.1. (Continued)

Option	Long form	Argument	Description
`-u`	`--log-usb-trfc`		Write a timestamped USB-traffic log
	`--usb-counts-only`		With `-u`: log receive timestamps and byte counts only, no hex dump — for throughput measurement
	`--diag-serial`		Log serial-channel troubleshooting detail, including the P2 download handshake steps
	`--ide`		IDE-integration mode (minimal UI)
	`--rts`		Use RTS instead of DTR for reset (overrides the per-device setting)
	`--console-mode`		Console output mode
	`--headless`		Run with no GUI (file logging only)
	`--timeout`	seconds	Exit after N seconds (**headless only**)
	`--end-marker`	[phrase]	Exit when `phrase` appears; with no value, matches `END_SESSION` / `DEBUG_END_SESSION`
	`--exit-on-end-session`		GUI batch: exit on the end-session marker, draining first
`-V`	`--version`		Print the version
`-h`	`--help`		Show help

Constraints. `-r` and `-f` are mutually exclusive. `--timeout` requires `--headless`. `--end-marker` requires `--headless` or `--exit-on-end-session`, and its phrase cannot be empty. The command line is validated before anything runs; any error is reported in full and the tool exits with code 2.

Examples.

RUN THIS

```
pnut-term-ts -n                                  # list devices
pnut-term-ts -r prog.bin -p P9cektn7             # RAM download (GUI)
pnut-term-ts -f prog.bin                         # FLASH download
pnut-term-ts -p P9cektn7 -u                      # enable USB logging
pnut-term-ts --ide --rts -p P9cektn7             # IDE mode, RTS reset
pnut-term-ts --headless -r TEST.bin --end-marker # headless, exit on marker
pnut-term-ts --headless -r TEST.bin --timeout 60 # headless, exit after 60s
```

Chapter 17: Keyboard Shortcuts

Application-level shortcuts (use `Cmd` instead of `Ctrl` on macOS):

Shortcut	Action
`Ctrl+R`	Start Recording
`Ctrl+P`	Playback Recording
`Ctrl+Q`	Exit
`Ctrl+,`	Preferences
`Ctrl+F`	Find in terminal
`F1`	Documentation

Cut / Copy / Paste use your platform's standard keys in any text field. Inside a debug window, mouse and key input may be forwarded to the running program when it asked for them (`PC_MOUSE` / `PC_KEY`); dragging a display window shows its position in the title bar (Chapter 8).

Chapter 18: Troubleshooting

The P2 is not detected. Check the USB cable and that the P2 is powered. Run `pnut-term-ts -n` to see whether it enumerates. Install FTDI drivers if needed and try another port. On Linux and macOS, check serial-port permissions (below).

Text is garbled or missing. Almost always a baud mismatch, and which way to fix it depends on where the binary came from. If you built with PNut or `pnut-ts` and passed `-b`, **try dropping it** — those images are auto-detected and carry their own rate (Chapter 6); look in the output for the line beginning `WARNING: --baud ... disagrees with this binary's compiled DEBUG_BAUD`, which names the mismatch outright. If you built with **any other toolchain**, or you are attaching to an *already-running* P2, there is no rate for us to read, so it is the opposite move: set the rate yourself with `-b`, or set the **Serial Baud Rate** preference — for the current project or for every project (Chapter 10) — if it is a board you come back to. Common rates are 115200, 921600, and 2000000.

If no rate makes it readable, stop trying rates: check the **framing**. PNut-Term-TS speaks 8N1 only (Chapter 7). A device expecting 7E1, or two stop bits, produces garbage at every baud rate there is. This never happens with a P2 — only when the tool is pointed at something else.

The download never finishes. The progress stalls and nothing is reported, because the P2's auto-baud never locked on and the chip is not answering. The usual cause is a link that cannot carry 2 Mbaud — a long lead, a marginal USB-serial adapter, or an FTDI clone. Lower the download rate and try again: `pnut-term-ts --downloadbaud 921600 -r myprogram.bin`. This is independent of your program's own output rate (Chapter 6); slowing the download does not slow the run.

The P2 does not reset or the program does not start. The reset control line may be wrong for your adapter. Set **DTR** or **RTS** for the device in PropPlug Management, or pass `--rts` for the session. Parallax PropPlugs use DTR; some clones use RTS.

A window is blank or data is missing. Confirm the program is actually running and sending `DEBUG ()` output. Open the Performance Monitor; if buffer usage is high, lower the data rate.

A window never opens, or the run stops with a display-name error (exit code 4). The name in the `DEBUG()` directive that creates the display is not one the tool can use — most often because it is also a directive keyword, such as `trace`. The check is case-insensitive, so `Trace` fails for the same reason. See *Naming a display* in Chapter 8; renaming the display in your P2 program and re-running is the whole fix.

The application feels sluggish under a heavy stream. Close the Debug Logger with **Window → Hide Log** — it is the most expensive window to draw, and closing it does not interrupt logging. **Window → Show Log** brings it back. Beyond that, close DEBUG windows you are not watching, or lower the data rate in the P2 program. A the "line(s) not shown" marker in the Debug Logger is the display deliberately keeping up, not an error and not data loss — the file has every line (Chapter 5).

A reopened Debug Logger looks like it is missing earlier output. The window repaints recent history, not the whole session; the file holds all of it. Open the log file itself to see everything.

Recording problems. Check free disk space and write permission to the recordings directory, and stop any recording in progress before starting a new one.

Platform notes.

- **Windows** — if port access is denied, make sure no other program holds the COM port; confirm the COM number in Device Manager.
- **macOS** — grant serial access if prompted; devices appear as `/dev/tty.usbserial-*`.
- **Linux** — add your user to the `dialout` group (`sudo usermod -a -G dialout $USER`, then log back in); devices appear as `/dev/ttyUSB*`.

Chapter 19: Support and Resources

Where the tools come from

Each address below is a **releases** page rather than any particular version, so it stays right as new builds appear.

- **PNut-Term-TS** — this tool.
  `https://github.com/ironsheep/PNut-Term-TS/releases`
- **`pnut-ts`** — the Spin2 / PASM2 compiler of Chapter 1.
  `https://github.com/ironsheep/PNut-TS/releases`
- **P2KB MCP** — the `p2kb-mcp` server of Chapter 15.
  `https://github.com/ironsheep/P2-Knowledge-Base-MCP/releases`

What you install there is the **server**, not the knowledge base. Once it is running, your assistant queries it and the P2 knowledge reaches you through the assistant — there is no knowledge-base download to go looking for, and no copy of it to keep current yourself.

The **Spin2 VS Code extension** is the one thing here with no download page at all. Get it from inside your editor: open VS Code's **Extensions** view, search for **`spin2`**, and install it. That is its formal name, and searching it is the whole procedure.

Reporting a problem, or asking

Two routes, and either is welcome. Use the issue tracker when something is wrong and you can describe it; use the forum when you would rather talk it through.

Issues — on whichever repository the trouble belongs to:

- `https://github.com/ironsheep/PNut-Term-TS/issues`
- `https://github.com/ironsheep/PNut-TS/issues`
- `https://github.com/ironsheep/P2-Knowledge-Base-MCP/issues`

Forum threads — each tool has one, and they are where announcements land:

- PNut-Term-TS — `https://forums.parallax.com/discussion/177897/new-pnut-term-ts-downloader-debug-application-for-p2-development-on-windows-mac-linux-yes-rpi`
- `pnut-ts` — `https://forums.parallax.com/discussion/175988/new-pnut-ts-compiler-for-p2-development-on-windows-mac-linux-yes-rpi`

If you run the serial line above 2,000,000 baud, the report the warning asks for goes to either of these (Chapter 6) — what you ran, on what platform, and whether anything went missing. That is how the verified ceiling gets raised.

Reading further

- **The `DEBUG()` display directives** — the official Parallax P2 DEBUG documentation, and the **P2 Debug Window Manual** for the display windows.
- **The single-step debugger** — the **P2 Single-Step Debugger Manual**.
- **Agentic P2 development** — **The P2 Architect's Guide, Part 3**, for the tool-chain workflow this guide's tool takes part in.
- **The Propeller 2** — Parallax's own documentation and the community-maintained references, and the Parallax Forums at `https://forums.parallax.com/`.